

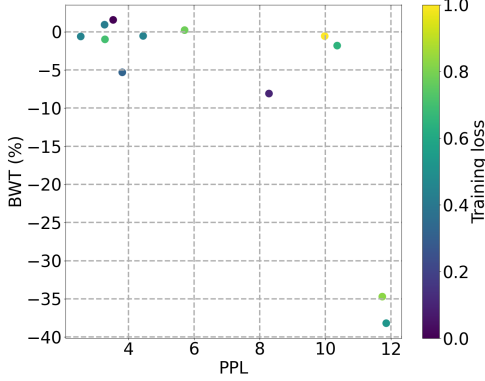


Figure 2: Fine-tuned Llama 3 8B Instruct on MBPP labels generated by various LLMs. Backward Transfer (BWT) measures performance drop on non-target tasks, showing that Low-perplexity labels training generally leads to less degradation (upper left) than ground truth (bottom right).

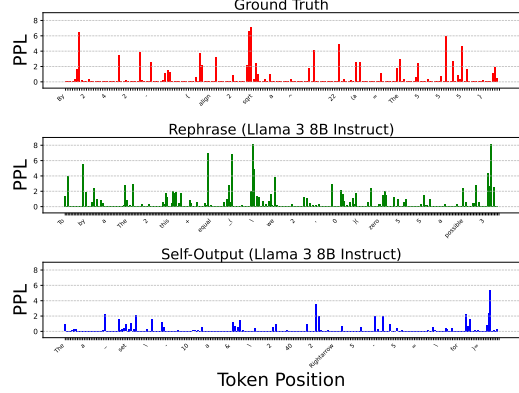


Figure 3: Comparison of token-level perplexity (PPL) distributions between human-annotated Ground Truth (top) and Llama 3 8B Instruct generations for Rephrase and Self-Output sequence (middle and bottom), where PPL of Self-Output data is low and with fewer spikes.

than the baseline fine-tuning in most cases, indicating much less degradation on non-target task capabilities. For more details, such as performance comparison with other models or other minor findings for self-output and rephrase method, please refer to Appendix D.

2.4 Analysis of perplexity across each sample

In this section, we analyze the perplexity of datasets generated by Self-Output and Rephrase to quantify their impact on model uncertainty and fine-tuning stability. We first collect the inference statistics of GPT-4o, GPT-4o mini, Gemma 2 IT 27B, Llama 3 8B Instruct, and Gemma 2 IT 2B from all three methods: Baseline Fine-tuning, Rephrase, and Self-Output strategy, along with their BWT, as Figure 2 shows that a higher perplexity of target training data could result in a higher degradation on non-target task. Second, we further collect the Llama 3 8B Instruct inference statistics from three kinds of training data: Baseline Finetuning (ground truth data), Rephrase, and Self-Output strategy. We evaluate the perplexity of responses by summing up the negative log probability of each token conditioned on its previous context in the sentence to measure the uncertainty of model outputs over the entire sequence. Specifically, the perplexity PPL^4 across a sequence is computed as:

$$PPL = -\exp\left(\frac{1}{N} \sum_{i=1}^N P(w_i | w_1, \dots, w_{i-1})\right), \quad (3)$$

where $P(w_i | w_1, \dots, w_{i-1})$ represents the conditional probability of token w_i given its preceding context, and N is the total number of tokens in the sequence.

Table 1 demonstrates that the Self-Output strategy achieves the lowest average perplexity of 1.16 in MBPP and 1.34 in MATH. Among the three approaches, Ground Truth exhibits the highest uncertainty with a mean perplexity of 4.83, and the perplexity of Rephrase is slightly higher than Self-Output. The lower perplexity of Self-Output suggests that LLMs are less likely to deviate far from their initial weights when fine-tuning on downstream tasks created by Self-Output. This characteristic maintains the model’s original performance on non-target tasks while effectively learning from the target task.

To investigate predictability at the token-level, we sample a single MATH response and visualize the token-level perplexity value. Figure 3 reveals distinct patterns across Ground Truth, Rephrase, and Self-Output responses. The first 20 tokens’ variance is 2.04 for Ground Truth, 2.31 for Rephrase, and 0.04 for Self-Output, which confirms our observation: Ground Truth exhibits frequent perplexity spikes of high amplitude (perplexity around 6-8), which reflects the natural variability in human problem-solving expressions. While Rephrase maintains similar peak amplitudes to Ground Truth,

⁴In this section, we use natural log instead of base 2 for computation convenience

i.e., more regular spike patterns, its variability is between Ground Truth and Self-Outputs. Self-Output demonstrates remarkably consistent low token perplexity values rarely exceeding 2, indicating highly predictable token sequences. These patterns suggest that Self-Output generates more deterministic solutions, potentially beneficial for maintaining consistent problem-solving approaches, while Ground Truth sequences introduce much higher variations. To demonstrate the different content of tokens distribution between Ground Truth and Self-Output responses, please refer to Appendix E.

3 Low-perplexity token learning via selective token masking

Our earlier analysis of Figure 3 reveals a notable discrepancy in token perplexity distributions between Self-Output generated data and other sources. This observation raises a central question: Is the superior performance of Self-Output methods primarily due to the *lower perplexity* of their generated tokens? To explore this hypothesis and design more efficient fine-tuning strategies, we propose Selective Token Masking (STM), a novel approach to supervised fine-tuning (SFT) that explicitly leverages token-level perplexity.

The core idea behind STM is simple yet effective: we use an existing instruction-tuned model to compute token-level perplexities and mask ground truth tokens whose perplexity exceeds a predefined threshold τ during training. Our investigation yields two key insights: First, STM highlights the pivotal role of token perplexity in shaping fine-tuning performance. Second, it shows that the gains observed from Self-Output methods can be replicated through perplexity-based token filtering, suggesting that performance improvements stem more from low-perplexity token distributions than from the self-generation process itself. STM thus offers not only computational efficiency but also a principled framework for understanding model adaptation during fine-tuning.

While our approach shares conceptual similarities with prior work such as [2, 20], it differs significantly in both implementation and efficiency. Previous methods typically involved a two-stage process: training a reference model on high-quality data to identify learnable (high-perplexity) tokens, followed by pre-training a larger LLM on this curated subset. In contrast, STM achieves similar goals in a streamlined single-stage workflow by directly using an existing instruction-tuned model to compute token perplexities in a single forward pass. This design avoids the need to train or maintain any additional models, significantly reducing computational overhead while preserving the benefits of low-perplexity training. This makes STM a compelling choice for efficient fine-tuning rather than full-scale pretraining.

Importantly, STM is orthogonal to other training strategies. It can be applied alongside different data sources (*e.g.*, ground truth or Self-Output) and training techniques (*e.g.*, full-weight fine-tuning or LoRA) to alleviate non-target task performance degradation. In the following sections, we explore how STM complements these various settings and contributes to more robust fine-tuning outcomes.

4 Experimental setting

To evaluate the effectiveness of our proposed method, we adopt benchmark datasets specifically selected to assess model robustness on both target and non-target tasks after fine-tuning. We include only datasets with verifiable ground truth, allowing us to filter out incorrect self-generated responses for a fair comparison with self-synthesized baselines and to ensure high data quality. Following recent findings [4] that LoRA [13] effectively mitigates performance degradation, we standardize our experiments by using LoRA for all fine-tuning. We then vary training strategies and data sources to study their impact on performance preservation. Details of our evaluation prompts and training configurations are provided in Appendix F. All implementation code and datasets are publicly available at: <https://github.com/appier-research/robust-llm-finetunes>.

4.1 Target and non-target datasets

We focus on three domains, programming, mathematics, and knowledge-based tasks, to study performance degradation after supervised fine-tuning with LoRA on each domain. For the programming and mathematics domains, we select MBPP and MATH for target task training, while others (MATH or MBPP test split, BIRD, GSM8K) and knowledge-based tasks for non-target task evaluation to assess generalization capabilities.

Table 2: Performance comparison across different models and target tasks in terms of task improvement (**TI**) on target test set and backward transfer (the degradation percentage, **BWT**) of non-target test set, and the cost of data related process before training (*e.g.*, generation of self-output, token PPL calculation and STM training criteria selection). Self-Output and our STM method have generally better BWT and TI, showing a better preservation of generalization capabilities after fine-tuning.

Model	Target task	Method	BWT(%)	TI(%)	Cost (GPU hours)
Gemma 2 IT 2B	MBPP	Baseline Fine-tuning	-38.19	-21.76	0
		Self-Output	-8.10	5.70	12 Hours
		Rephrase	-3.23	-4.69	30 Minutes
		STM _{$\tau=2.5$} (Ours)	0.42	0.00	5 Minutes
	MATH	Baseline Fine-tuning	-36.68	-22.78	0
		Self-Output	-1.73	9.06	≥ 2 Days
		Rephrase	-14.06	-28.83	39 Minutes
		STM _{$\tau=2.5$} (Ours)	-2.93	7.83	8 Minutes
Llama 3 8B Instruct	MBPP	Baseline Fine-tuning	-34.71	-2.23	0
		Self-Output	3.09	1.55	16.8 Hours
		Rephrase	-5.32	-9.58	36.8 Minutes
		STM _{$\tau=2.5$} (Ours)	-0.16	3.20	4.5 Minutes
	MATH	Baseline Fine-tuning	-14.12	-17.83	0
		Self-Output	0.31	9.55	≥ 2 Days
		Rephrase	-1.09	4.78	29.3 Minutes
		STM _{$\tau=2.5$} (Ours)	-0.30	6.37	7 Minutes

Programming. For code generation evaluation, we split MBPP into target task training data and non-target task testing data, while BIRD serves as a non-target assessment. This pairing tests both direct programming ability and cross-language generalization from Python to SQL.

Mathematical reasoning. We split MATH into target task training data and non-target task testing data, and GSM8K for non-target task testing. These datasets differ in format: MATH features competition-style problems, while GSM8K uses natural language, allowing us to assess generalization across mathematical expression styles.

Knowledge-based. To assess broader generalization capabilities beyond mathematical and programming domains, we incorporate ARC-Challenge, a subset of the ARC science question dataset.

For details about the data curation of the target and non-target datasets, please refer to Appendix A.

To explore more performances of non-target tasks including instruction following and safety performance under limited model choices, please refer to Appendix B.

5 STM results

Table 2 shows that with a threshold to filter out high perplexity tokens, such as 2.5, which approximately filters out around 20% to 24% of total tokens, its **BWT** is close to 0%, and STM also achieves comparable **TI** to the Self-Output method. It indicates that STM not only improves target task performance but also reduces non-target task degradation. This result agrees with our hypothesis that the presence of high perplexity tokens is one of the causes of performance degradation of the ground truth model, and low perplexity tokens sufficiently mitigate such negative impact for strong generalization without the need for self-generated data. We also applied STM to other models, and the results can be found in Appendix D. Our conclusions are still the same across different models.

5.1 Does filtering high perplexity tokens always perform better?

From the intuition of the high perplexity filtering mechanism of STM, we have done an ablation study on the masking criteria of perplexity by filtering out high, random, and low ppl tokens. More specifically, high ppl refers to filtering the top 25% ppl of all tokens, random filtering refers to filtering the randomly selected 25% of all tokens, and low ppl refers to filtering the lowest 25% ppl of all tokens from the dataset. As the upper part of Table 3 shows, masking high ppl tokens in STM simultaneously reduces performance degradation and improves target task performance. Likewise, we have the same conclusions for Llama 3 8B Instruct on such ablation test, please refer to Appendix D.